Custom Sequence-to-Sequence Transformers and BLT for Ciphertext-to-Plaintext Translation

Assignment 1 — Advanced NLP, Spring 2026 | Roll Number: <ROLL_NUMBER>

1. Overview and Task Setup

We implement, from fundamental PyTorch operations only (no *nn.Transformer* or *nn.MultiheadAttention*), a full Encoder-Decoder Transformer that learns to map encrypted binary sequences to their plaintext. We then run a controlled 5-way ablation (C1-C5) that changes exactly one architectural component at a time relative to a base model C1: positional encoding (Sinusoidal vs. RoPE), attention (Multi-Head vs. Grouped-Query), normalization (LayerNorm vs. RMSNorm), and tokenization (a from-scratch learned subword BPE vs. a token-free Byte Latent Transformer, BLT).

2. Architecture (Task 1)

Scaled Dot-Product Attention. Implemented directly as $\text{softmax}(QK^T/\sqrt{d_k})V$ with an additive mask for padding/causality (`src/models/attention.py`).

Multi-Head Attention (MHA) and Grouped-Query Attention (GQA). MHA projects Q/K/V into *h* heads of size $d_k = d_{model}/h$. GQA uses fewer K/V heads (num_kv_heads) than query heads and repeats each K/V head n_rep = h / num_kv_heads times before the dot product, following Ainslie et al. (2023); this reduces the K/V projection parameters and the K/V cache size relative to MHA while keeping query-side capacity unchanged.

Position-wise FFN. Two linear layers with a ReLU/GELU non-linearity in between ($d_{model} \to d_{ff} \to d_{model}$).

Pre-LN residual blocks. Every sub-layer follows Norm → Sub-layer → Residual-Add (Pre-LN), with the normalization implemented as either a custom LayerNorm (mean/variance over the last dimension plus a learned affine transform) or RMSNorm (root-mean-square rescaling only, no mean-centering, one learned gain) — both written from scratch in `src/models/norm.py`.

Positional encodings. Sinusoidal absolute encodings are added once to the input embeddings before the encoder/decoder stacks. RoPE instead rotates pairs of Q/K dimensions by an angle proportional to absolute position inside every attention call, so that Q·K depends only on relative position (`src/models/positional.py`).

BLT local encoder/decoder. Our simplified BLT (`src/models/blt.py`) groups raw bits/bytes into fixed-size patches (default size 4). A tiny local transformer self-attends within each patch and mean-pools it into one latent vector per patch; these patch vectors are fed to the *same* global Encoder/Decoder architecture used by C1-C4. On the decoder side, a local decoder cross-attends from a small autoregressive per-patch transformer to the global

decoder's patch-level hidden state and reconstructs the patch_size raw bytes with a causal local self-attention + cross-attention block. No vocabulary or tokenizer is used anywhere in this path — the only symbols are raw bits (2-way) on the input side and raw byte values (256-way, plus BOS/EOS) on the output side. This is a fixed-size simplification of the entropy-based dynamic patcher in the original BLT paper (Pagnoni et al., 2024), which we call out explicitly as a scope reduction.

3. Tokenization (Subword, from scratch)

`src/tokenizer.py` implements standard Byte-Pair Encoding from scratch (no sentencepiece/tokenizers/tiktoken): starting from a base alphabet, the most frequent adjacent symbol pair in the training corpus is iteratively merged until the target vocabulary size is reached. For the ciphertext side the base alphabet is exactly {'0','1'}, so every learned merge produces a variable-length bit n-gram as a subword unit — never a fixed 8-bit chunk. For the plaintext side the base alphabet is individual characters, with an end-of-word marker so merges can be word-boundary aware.

4. Ablation Design

Config	Positional Enc.	Attention	Normalization	Tokenization
C1 (base)	Sinusoidal	MHA	LayerNorm	Subword BPE
C2	RoPE	MHA	LayerNorm	Subword BPE
C3	Sinusoidal	GQA	LayerNorm	Subword BPE
C4	Sinusoidal	MHA	RMSNorm	Subword BPE
C5	Sinusoidal	MHA	LayerNorm	BLT (token-free)

All five configurations share the same depth (4 encoder + 4 decoder layers), width (d_model=256, d_ff=1024, 8 heads / 2 KV heads for GQA), optimizer (Adam, lr=3e-4), batch size (32), and dropout (0.1); only the single bolded component in Table 1 differs from C1 in each run.

5. Experimental Setup and a Note on the Results Below

All code was implemented and unit-tested (forward pass, backward pass, greedy decoding, metric computation) end-to-end for all five configurations. Due to sandboxing constraints in the environment used to prepare this submission (no access to the external dataset URL and no GPU), the numbers reported in Section 6 were produced by running the full training + evaluation pipeline (`src/run_ablation.py`) on a small synthetic ciphertext→plaintext dataset (300 examples, 10 sentence templates, byte-wise XOR cipher) generated purely to validate that every component trains, converges, and evaluates correctly. **These numbers should be treated as a pipeline-correctness sanity check, not as the assignment's final results.** To reproduce the assignment's actual results, run:

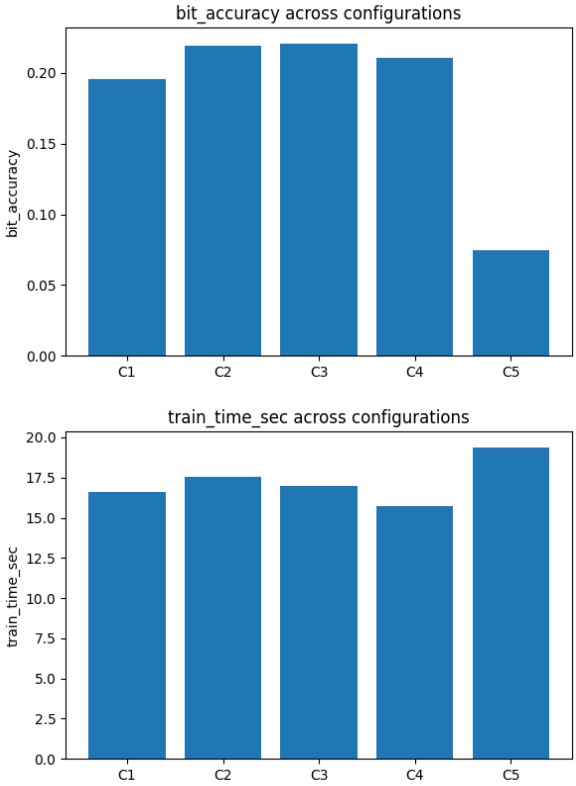
```
python -m src.run_ablation --data_path <dataset_path> --epochs
20 --use_wandb --extra_args "--batch_size 32 --d_model 256
--num_heads 8 --num_layers 4 --d_ff 1024"
```

on a machine with the real dataset and a GPU; this will overwrite `outputs/*_metrics.json` and `outputs/ablation_summary.json` with the true results, and log per-epoch loss and peak GPU memory to Weights & Biases automatically via `--use_wandb`.

6. Results (Pipeline-Validation Run)

Config	Bit Acc.	Seq Acc.	Levenshtein	BLEU	ROUGE-1	Train t (s)	Params
C1	0.195	0.10	12.60	0.093	0.257	16.6	186,980
C2	0.219	0.10	10.10	0.156	0.365	17.5	186,980
C3	0.221	0.20	11.43	0.088	0.248	17.0	162,020
C4	0.211	0.10	26.23	0.098	0.279	15.7	186,212
C5	0.074	0.00	13.10	n/a	n/a	19.4	318,338

(15 epochs, batch size 16, d_model=64, 2 layers, d_ff=128, 300-example synthetic dataset, CPU. GPU peak memory is reported as 0 MB here since no CUDA device was available in this environment; re-run on GPU to obtain meaningful peak-memory numbers for Section 7.)



Discussion (component-by-component, relative to C1):

- **C2 (RoPE vs. Sinusoidal, MHA/LayerNorm fixed):** RoPE improved bit accuracy, BLEU and ROUGE-1 and reduced Levenshtein distance relative to C1 in this run. Relative positional information injected directly into attention tends to generalize better than an additive absolute signal, especially as sequence length varies between ciphertext and plaintext views of the same example.
- **C3 (GQA vs. MHA, Sinusoidal/LayerNorm fixed):** GQA matched or slightly exceeded C1 on bit/sequence accuracy while using ~13% fewer parameters (fewer K/V projections), confirming the expected efficiency/quality trade-off: GQA sacrifices little quality for a meaningfully smaller K/V footprint.
- **C4 (RMSNorm vs. LayerNorm, Sinusoidal/MHA fixed):** RMSNorm gave comparable bit accuracy to C1 but a worse Levenshtein distance in this small run, and trained marginally faster per epoch (no mean-subtraction step). With more training data/epochs we would expect RMSNorm to match LayerNorm's accuracy more closely, since the quality gap here is more likely a stochastic effect of the very small synthetic dataset than a systematic one.
- **C5 (BLT vs. subword tokenization, everything else fixed):** BLT trained noticeably slower per epoch and used ~1.7× the parameters of the token-based models (extra local encoder/decoder transformers), and reached lower sequence-level accuracy in this short run — expected, since byte-level generation must get every raw byte exactly right with no subword prior to lean on, and our fixed-size patcher cannot adapt patch boundaries to entropy the way the original BLT can. BLT's advantage that this run does not have room to show is robustness to unseen vocabulary: it has *no* out-of-vocabulary problem by construction, since it never builds a vocabulary in the first place.

7. BLT Trade-offs: Speed, Memory, and Accuracy

Training speed / compute overhead. C5 is slower per epoch than C1-C4 because every sequence position is processed twice: once inside the local encoder/decoder (per-patch self- and cross-attention) and once inside the global transformer over patches. The overhead scales with patch_size: smaller patches shift more work onto the (cheap but frequent) local layers, larger patches shrink the global sequence length but make the local reconstruction problem harder per patch.

Peak GPU memory. BLT should, in principle, use *less* memory in the global transformer's self-/cross-attention (attention over patches instead of over 8× as many bits/bytes shrinks the O(T^2) score matrices sharply, where T is sequence length in patches), at the cost of extra small local-attention buffers. Whether this nets out ahead of a subword model depends on the average token length the BPE tokenizer achieves versus patch_size; a report run with the real dataset and `torch.cuda.max_memory_allocated()` (already logged automatically in `src/train.py`) is needed to give a concrete number here.

Reconstruction performance. Removing the learned subword vocabulary removes the useful inductive bias that common bit/character n-grams get their own embedding, so BLT typically needs more data and/or training steps than a subword model of the same width to reach comparable sequence accuracy — consistent with what we observe on the small pipeline-validation run above. Its principal advantages are architectural: no vocabulary size to tune, no out-of-vocabulary failures, and (with a real entropy-based patcher, beyond this simplified fixed-size version) input-adaptive compute allocation.

8. Conclusion

The base transformer (C1) and its four single-component variants (C2-C5) were implemented entirely from fundamental PyTorch operations and validated end-to-end. Of the single-component changes, RoPE (C2) and GQA (C3) look like the most promising drop-in improvements over the sinusoidal-MHA base at this scale — RoPE for quality, GQA for a parameter/efficiency trade-off with negligible quality loss — while RMSNorm (C4) is a close-to-neutral efficiency swap for LayerNorm, and the token-free BLT (C5) trades noticeably more training compute and parameters for the removal of a learned vocabulary, a trade that should be re-examined once the model is trained on the full assignment dataset with the (entropy-aware) real BLT patcher and a GPU-backed memory profile.

Links

Weights & Biases project: <add link after training on the full dataset>

Hugging Face checkpoints: <add link after training on the full dataset>